Methodology

The end-to-end road we took once we had **365 days of hospital dispensing records**, written as a human walk-through rather than an auto-generated spec. Every step explains *what* we did, *why* we did it, and *what we would have done differently with more time*.

0 · The starting point

We received a year of records from a Lahore hospital with one row per medicine line on each admission. The columns were:

Column	What it holds
`MRN`	patient medical-record number
`Patient Name`, `Age`, `Gender`	demographics
`DOA`	date and time of admission (string)
`Diagnosis`	free-text diagnosis (~150 unique strings, many spelling variants)
`GenericName`	molecule (~85 unique, again with case / spelling variants)
`Tradename`	brand name as dispensed

So one *real-world* admission → many rows (one per medicine in the basket). The seed file the project started with was 45 days, ~1,000 rows. The current `data.csv` is one full year, ~31,000 rows — and was extended deterministically using `scripts/synthesize_year.py` to keep us inside realistic dengue / Eid / monsoon / smog patterns instead of a flat extrapolation. That synthesizer is documented in §1.

1 · Data understanding & cleaning

The first thing we did was *look* at the data, not model it.

1. **Parsed `DOA` to a datetime.** Mixed `M/D/YY` formats forced

`pandas.to_datetime(..., errors='coerce')` and we dropped the rows where parsing failed.

1. **Normalised `GenericName`.** Upper-cased, stripped, collapsed

variants like `OMEPRAZOLE` / `Omeprazole` / `OMEPRAZOL` into one key. This single step changed our medicine count from 85 to 17 — a clean signal that name-cleaning matters more than fancy modelling for this data.

1. **Built a daily panel.** For every (date, medicine) pair we counted

rows. Then we re-indexed against the full date range so missing days became explicit zeros. Without this step the rolling-mean features lie about quiet weeks.

1. **Spotted dengue / viral fever cases** by string-matching the

`Diagnosis` column against `("DENGUE", "DHF", "DSS", "FEBRILE ILLNESS", "VIRAL ILLNESS", "VIRAL FEVER", "PYREXIA", "PUO")` — these formed our `dengue_proxy_daily` series, used both as a feature signal and as a sanity check on the seasonal curve.

1. **Plotted everything we built** before training a single model —

per-medicine line charts, a calendar heatmap, weekday boxplots, a histogram of admissions per day. This is what the *Preprocessing chart* on the dashboard exposes to the end user (rolling Mean / Median / Mode / Std / Variance overlays at 7/14/28-day windows). If we had skipped this step we would not have caught the weekend dip (Saturday/Sunday emergency-only) or the Friday OPD half-day, both of which became calendar features.

When the seed CSV was only 45 days

We needed a year to train against. The pragmatic choice was a *deterministic event-driven generator* (`scripts/synthesize_year.py`) that:

- Re-uses the real patient pool (real MRNs + a sampled extension)
- Defines clinical scenarios (`DENGUE_FEBRILE`, `GASTROENTERITIS`,

`RESPIRATORY_LRTI_URTI`, `HEAT_DEHYDRATION`, `TRAUMA_RTA`, `CARDIO_NEURO`, `MIGRAINE_MSK`, `APD_GASTRITIS`)

- For each day, multiplies a baseline visit count by event modifiers

(dengue intensity, monsoon, smog, heat, Eid, Ramadan, weekend, …)

- Picks medicines from a clinically plausible basket per scenario
- Repeats fluid lines (NS) 1-3× per visit to match real dispensing

realism

The synthesised year is honest about being synthetic — it is *not* labelled real data. Once a hospital provides a true year, you re-run `model.py` on the real CSV and the synthesizer is no longer needed.

2 · Feature engineering

2.1 Calendar

Feature	Why
`dayofweek`, `is_weekend`, `is_friday`	Saturday / Sunday are emergency-only; Friday is half-day in PK. Weekly cycle is the strongest pattern in the data.
`month`, `day`, `weekofyear`, `dayofyear`	annual seasonality
`dow_sin/cos`, `month_sin/cos`, `doy_sin/cos`	smooth cyclical encodings — the boundary between Dec 31 and Jan 1 should not be a feature cliff

Feature	Why
`is_monthstart`, `is_monthend`	pay-day proximity

2.2 Cultural / national

We hand-curated the Hijri windows for 2024-2027 because automated Hijri libraries on Mac/Linux disagreed by a day or two. The features are:

- `is_ramadan`, `is_eid_fitr`, `is_eid_adha`, `is_muharram`
- `days_to_eid_fitr`, `days_to_eid_adha` (absolute distance)
- `days_to_ramadan` (signed — positive = upcoming, negative = past)
- `is_public_holiday` against a hard-coded list of Pakistan public

holidays

- `inflation_idx` — Pakistan CPI YoY % proxy keyed by `YYYY-MM`

2.3 Lahore weather & dengue

Real-time weather data was out of scope, so we used:

- **Climatology** for Lahore — monthly mean temperature (PMD 1991-2020

normals), monthly rainfall, and PM2.5 / smog AQI (PCAP / IQAir).

- **Dengue** — a Gaussian seasonal curve centred on **DOY 288 (Oct 15)**

with σ=28 days, scaled to [0,1]. This matches NIH Pakistan epidemiological reports (2022-2024) and is independent of the year, so it works for forecast dates too.

These are climatology *not* observations. See "Limitations" below.

2.4 Lag & rolling

For every `(medicine, date)` row:

- `lag_1`, `lag_7`, `lag_14`, `lag_28` — yesterday, last week's same day,

last fortnight's, last month's

- `roll7_mean / roll14_mean / roll28_mean` — rolling averages of past

demand (shifted by one day to avoid leakage)

- `roll7_std`, `roll28_std` — rolling volatility

The 28-day lag was the single biggest accuracy unlock — it lets the model see the monthly billing / restocking cycle.

2.5 Per-medicine context

We also pass `medicine_id` (an integer) and `medicine_avg` (a global mean per medicine). Without these the model would have to learn each medicine's level from the lag features alone, which fails for new medicines or quiet stretches.

3 · Model selection

We deliberately tried multiple model families on the same held-out window so we could *report* their behaviour rather than guess.

3.1 Validation strategy

A random 80/20 split would leak future days into training. We used a **time-based** split — the last 20 % of dates becomes the test window for tabular models, and a per-medicine 80/20 cutoff for the univariate ones. The same metric set runs on every model so the comparison table is apples-to-apples:

- **MAE** — mean absolute error in raw units
- **RMSE** — penalises large misses
- **R²** — coefficient of determination
- **Confidence%** — share of held-out days where prediction is within

±25 % (or ±2 units) of actual. We invented this metric because R² is abstract for a procurement officer; "Confidence 73 %" is read as *"7 days out of 10 we're within a quarter of the true demand"*.

3.2 Models tried

Model	Why we tried it
HistGradientBoosting (sklearn)	strong default for tabular features, fast, handles missing values
RandomForest (sklearn)	low-variance baseline; we wanted a non-boosted comparator
XGBoost	gold-standard tabular; has libomp dependency on Mac
ARIMA(2,1,2) (statsmodels)	classical univariate baseline
SARIMA(1,1,1)(1,1,1,7) (statsmodels)	weekly seasonality on top of ARIMA
Prophet (additive)	popular, captures yearly + weekly + holiday effects
LSTM (PyTorch)	recurrent baseline — sequence length 28, hidden 24, 20 epochs, single-threaded

3.3 What we learned from the bake-off

Final scores on the held-out window:

Model	R^2	Confidence	MAE
SARIMA	0.81	73%	1.80
RandomForest	0.80	74%	1.72
HistGradientBoosting	0.79	73%	1.79
ARIMA	0.78	73%	1.82
XGBoost	0.77	70%	1.89
LSTM	0.74	68%	2.01
Prophet	0.64	65%	2.35

Take-aways:

- The tabular models and SARIMA are within ~3 % of each other on R^2.

When the gap between models is this small, **the right move is to ship the comparison table** to the user, not pretend one is obviously best.

- LSTM under-performed despite the cost — typical for short, noisy,

highly-seasonal series. Better implementations (longer training, larger hidden state, better regularisation) might close the gap, but the marginal R^2 is unlikely to be worth the runtime.

- Prophet's additive seasonality didn't fit the dengue spike well; its

residuals showed it under-shoots Sep-Oct.

3.4 Forecast horizon

The model trains once and produces a 180-day daily forecast. The dashboard then lets the user slice that into **3 / 4 / 5 / 6 month** horizons — no re-training. This was a conscious trade-off: bigger horizons accumulate recursive-forecast error, so we cap at 6 months.

4 · Explainability

4.1 Global

We use **permutation importance** (sklearn `permutation_importance`, n_repeats=3) instead of TreeSHAP because `HistGradientBoosting`'s internal structure is not a friendly target for the standard SHAP explainer. Permutation has the same intuition — "shuffle this column, how much does the score drop?" — and works on any model.

4.2 Local

For each top medicine we compute a **finite-difference contribution** around its latest row: nudge each feature toward its column mean, measure the change in the predicted value. The 8 features with the largest absolute change are surfaced as the LIME-style local explanation.

4.3 Per-medicine event impact

Beyond model-level explanations, we built a *medicine* × *event* table that the user actually wants:

- For each medicine and each event window (`dengue_peak`, `dengue_season`,

`monsoon`, `summer_heat`, `smog_season`, `ramadan`, `eid_fitr`, `eid_adha`, `muharram`, `weekend`):

- **historical avg/day inside that window**
- **uplift % vs. that medicine's overall baseline**
- **forecast totals for the next instance of the window**, computed

per horizon (3/4/5/6 m) so the filter on the dashboard is instant.

This is the "why does forecasting matter" answer for a procurement officer: *"Paracetamol is +80 % during dengue peak — order an extra 500 units for October."*

5 · Dashboard

The dashboard is a static Next.js 14 app — every route is rendered as HTML at build time (`force-static`). At runtime there's no Python, no API call, no DB. It is just a JSON file (`forecasts.json`) shipped with the build.

Component	What it answers
KpiCards	data range, MAE, RMSE, top medicines
HistoryChart	total daily admissions (the "shape" check)
PreprocessingChart	per-medicine daily line + rolling Mean/Median/Mode/Std/Variance
CalendarHeatmap	GitHub-style daily grid, instantly shows seasonality
ForecastSection	horizon toggle (3/4/5/6m) + model selector
■ ModelComparison	sortable table of seven models with R^2 + Confidence + MAE/RMSE
■ ForecastChart	per-medicine daily forecast line, monthly tags swap with model
■ MonthlyHeatmap	medicine × month forecast totals
■ MedicineFeatureImpact	medicine × event uplift / forecast table
LahoreTrendsChart	dengue × temp × rain × AQI × Ramadan × Eid overlay
TopMedicinesTable	demand-sorted list with sparkline links
ShapChart	global feature importance bar chart
LimePanel	per-medicine local contributions
MacroPanel	static reference cards (calendar, climatology, …)

Why static?

Hospitals are usually OK with a refresh cadence in days, not seconds. A static export removes a whole class of operational issues — no server to monitor, no auth tokens to rotate at runtime, deploys are diffs of one JSON file. When real-time becomes a requirement we can flip to the FastAPI backend that already exists in `backend/`.

6 · Limitations (today)

The most honest section in the document.

1. **Single hospital, one year of data.** No multi-site or multi-year

priors; we cannot validate dengue-peak Oct 2026 against Oct 2024 numbers because we do not have them.

1. **Lahore weather & AQI are climatology, not observations.** Monthly

means smooth out the actual smog spikes, monsoon bursts, and heat waves that drive admissions in real time.

1. **Dengue intensity is a Gaussian curve, not a feed.** The 2022 and

2023 outbreaks were both more violent than a "typical" October — our curve will under-shoot a bad year and over-shoot a quiet one.

1. **The seed CSV was 45 days; the current dataset is synthesised.**

Patterns are clinically plausible (dengue, Eid surges, smog, etc.) but they are not real epidemiology. Re-train on a real year before shipping to a hospital.

1. **Recursive forecasting accumulates error.** Past ~90 days the

per-day projections lean increasingly on previous predictions (lags), which compounds bias. We expose 3-6 m for this reason and not 12 m.

1. **No medicine-substitution modelling.** If `ondansetron` runs out

the basket usually shifts to `metoclopramide` — our model treats them as independent series. A real procurement system would tie them.

1. **No price elasticity / inventory cost.** The model forecasts

units, not money or stock-out probability. The CFO question is adjacent, not solved.

1. **Confidence% is a coverage proxy, not a calibrated interval.** A

prediction of 10 ± 25 % does not tell you the chance of seeing 12. For that we need quantile regression or conformal prediction.

1. **No real-time data ingest.** Today the workflow is "drop a CSV,

re-run model.py, push". Hospital systems expect HL7 / FHIR feeds, which we do not consume.

1. **No authentication or multi-tenant separation.** Anyone with the

URL sees the dashboard. For a real deployment we need per-hospital data isolation and role-based views.

7 · Future plans

Roughly in the order we'd take them:

7.1 CSV upload from the UI

A drop-zone on the dashboard accepts a fresh year of dispensing data, runs validation (column presence, date parsing, medicine normalisation), queues a re-train, and live-updates the dashboard once `forecasts.json` is rewritten. Today this is a manual `python model.py && git push`.

7.2 User authentication & multi-tenant

- Email/password + OTP via NextAuth (Auth.js) or Clerk
- Per-hospital data isolation — each tenant has its own CSV upload,

its own `forecasts.json`, its own dashboard

- Role-based views:
- **Procurement** — full forecast + order recommendations
- **Pharmacist** — daily medicine view + substitutions
- **Admin** — uploads, retrains, audit log

7.3 Live signals

- **Weather / AQI** — pull live Lahore data from PMD or IQAir into the

feature pipeline (with a graceful fall-back to climatology).

- **Dengue counts** — scrape NIH Pakistan weekly bulletins and feed

real case counts as a feature, not just a sinusoid.

- **Inflation / FX** — SBP CPI feed instead of a hard-coded dict.

7.4 Inventory integration

Pair forecasted demand with current stock to produce a **reorder-point report**. For each medicine: forecast horizon vs. current stock vs. lead time → "order N units by date X to avoid stock-out".

7.5 Per-prediction confidence intervals

Add a quantile-regression head (or apply conformal prediction to the existing point predictions) so each forecast is `(low, point, high)` — useful for safety stock sizing.

7.6 Model-blend

Today the dashboard picks one model. A weighted blend (e.g. weight SARIMA + RandomForest + HGB by per-medicine validation skill) usually improves the average and almost never hurts it. Cheap to add once the model comparison infrastructure exists, which it now does.

7.7 Mobile-first procurement view

A trimmed, phone-friendly view showing only the next-30-day procurement list with a single-tap "approve / re-order / hold" workflow, intended for use during the morning standup.

7.8 Alerts & notifications

Email/Slack a procurement lead when the model forecasts a >50 % spike in any medicine over the next 14 days, or when the dengue intensity curve crosses 0.5.

7.9 Audit & versioning

Every retrain stamps `forecasts.json` with the input CSV hash, model version, and a delta vs. the previous run, so the team can answer "why did this number change" without re-deriving it.

7.10 Real-time data ingest

HL7 / FHIR feed support for hospitals that have a modern HIS, so the CSV is built from the live admissions stream rather than a manual export.

8 · Reproducing this work

```
# 1. install Python deps (requires libomp on Mac for xgboost)
brew install libomp                    # Mac only
pip install -r requirements.txt

# 2. train + score every model + write forecasts.json
python model.py
#    runtime: ~5-10 min on a laptop CPU
#    output:  frontend/public/forecasts.json (~1 MB)
#             backend/forecasts.json

# 3. start the dashboard
cd frontend
npm install
npm run dev                             # http://localhost:3000

# 4. (optional) start the API
cd ../backend
uvicorn main:app --reload --port 8000   # http://localhost:8000/docs

# 5. (optional) re-bootstrap the synthetic year of data
python scripts/synthesize_year.py
```

That's it. Everything in this document is reflected in the code; if the code says one thing and the document says another, treat the code as authoritative and open a PR to fix the document.